

What the market is asking for

Seventeen features the plan does not have, each with the page it came from. Research round of 18 September 2026.

38

sections

8.4k

words

0

screens

print

18 Sept 2026

Contents

00	What this is, and how to read it
PART ONE. FIVE FACTS THAT MOVED WHILE WE WERE PLANNING	
01	1 Agents now read more than people do
01	2 Almost nobody lets an agent publish unreviewed
01	3 Writing documents is what people use AI for
01	4 The instruction file is now a mass-market artefact
01	5 Spec-driven development is enormous in attention and thin in practice
01	6 The markdown file in the repository is beating the purpose-built mechanism
PART TWO. SOMEBODY IS ALREADY STANDING ON OUR GROUND	
02	1 Ritemark took our sentence
02	2 OpenMarkdown built the agent bridge, and handed us the design
02	3 OpenKnowledge gave us our own marketing line, by accident
02	4 Zed Delta went to public beta two days ago, and it is the serious one
02	5 What none of them did
PART THREE. THE FEATURES, RANKED	
13	T1. The MCP server moves out of Later, and it proposes rather than writes
14	T2. The instruction-file set becomes the headline of S11, not a tab in it
15	T3. Open a spec folder as one document
16	T4. The published page serves a markdown twin and an llms.txt
17	T5. Stamp the accept into the front matter, using Google's field names
18	T6. The problems panel becomes the bundle, and the bundle is the pitch
19	T7. Front matter is the most-wanted thing on the largest markdown forum, and it is our name
20	T8. Suggesting mode, over the queue we already have
21	T9. Comments anchored to a byte range, batched into one revision request
22	T10. Staleness: anchor a paragraph to code, and say when the code moved
23	T11. Rewind the document and the agent's context together
24	T12. Shared attention: reveal, and wait for the person
25	T13. Review from the phone
26	T14. A live size readout on an instruction file, and nothing more
27	T15. Draft, then promote, as a visible state on the file

28 T16. A multi-file change that arrives as a reviewable set

29 T17. A command line, because in 2026 the thing typing the command is an agent

PART FOUR. WHAT WE MUST STOP SAYING

04 1 The change queue is not novel, and saying it is invites an easy rebuttal

04 2 We are not first to review markdown prose

04 3 The headline is taken

04 4 Do not clone iA Writer's annotations

04 5 We retired an idea on a reason its own evidence does not carry

04 6 The thing we should say instead

PART FIVE. WHAT THIS DOES TO THE PLAN

05 1 The seven moves

05 2 What it costs

05 3 The one question this raises for the founders

How to read this guide

Studio Zephyrus. 18 September 2026. For Sagnik and Amit, to read before the build starts.

§0 What this is, and how to read it

This is a research round against one question. **What are people asking for, in the agent era, that frontmatter does not have, that we could build, and that would help us sell?**

- Nine research branches ran on 18 September. **176 findings, about 103,000 words.** Every one carries the page it came from and the date it was opened.
- Plus five things measured directly in this session, against live registries and APIs.
- The plan of record is `docs/mvp0/PRODUCT-PLAN.md`, revision 6. Anything already in it is not reported here as a gap.

The signal tags are the plan's own.

TAG	MEANING
[M]	A page was opened and the string quoted
[O]	Measured in this session, from a live source
[R]	Earlier research of ours
[P]	Follows from a decision already taken
[Z]	A founders' decision

The short version, in six lines.

1. The plan is a very good markdown editor that has AI in it. The market is asking for **the place where a person and their agents meet over a document.**
2. The two things that would make that real, the **MCP server** and the **public API**, are both sitting in our **Later** column, behind eight phases.
3. Somebody shipped the obvious version of our positioning in July, free. Two more in June and August.
4. All three skipped the hard part, which is the part we already built.
5. **91 per cent of teams will not let an agent publish unreviewed**, and the only working review surface in the category is a pull request.
6. Our own research has eleven things in it that never reached a screen, and four contradictions.
7. Seventeen features are proposed here. Seven change the plan. None of them changes what frontmatter is.

PART ONE. FIVE FACTS THAT MOVED WHILE WE WERE PLANNING

§1 1 Agents now read more than people do

- [M] Mintlify published its State of Knowledge report on **16 September 2026**, two days ago, from 329 respondents.
- In August 2026, across their sites, agents made **257 million requests** against **131 million** human page loads. Close to two to one.
 - **83 per cent** of that agent traffic arrived through machine-friendly routes: markdown versions of pages, `llms.txt`, and agent skills.
 - Adding a link to `llms.txt` inside the markdown version of a page was a **twentyfold** improvement over HTML in their 2,400 controlled tests, and cut token use by up to 60 per cent.
 - **7 per cent** of companies have made every surface they own readable by an agent.

What it means for us. Our published page and portfolio are described nowhere as serving a markdown twin or an `llms.txt`. If you publish from frontmatter today, the larger half of your readers cannot read you properly.

§1 2 Almost nobody lets an agent publish unreviewed

[M] Same report, and this is the number the whole product rests on.

"83% of respondents say AI agents now draft documentation updates for their team. But humans haven't left the loop, only 9% let agents publish without review."

- So **91 per cent insist on a review step**, and the only working review surface in the category is a pull request.
- [M] Mintlify's own telemetry: **367,000** agent-directed documentation updates between February and August 2026, 95 per cent of them fired by webhook or cron, 61 per cent of the resulting pull requests merged.

What it means for us. Our central bet stops being a belief and becomes a measured preference. It also tells us who the customer is. A person on a documentation team who does not use GitHub is reviewing agent output in a pull request, or not at all.

§1 3 Writing documents is what people use AI for

[M] Stack Overflow's developer survey, AI section. Note the date: the 2026 edition is not published, so these are the **2025** figures and must be dated as such.

TASK PEOPLE MOSTLY USE AI FOR	SHARE
Documenting code	30.8%
Creating or maintaining documentation	24.8%
Debugging or fixing code	20.7%
Writing code	16.9%
Committing and reviewing code	10.2%

And the frustration, in the survey's own words, on 25,332 responses:

"The biggest single frustration, cited by 66% of developers, is dealing with 'AI solutions that are almost right, but not quite.'"

On trust, on 12,941 responses: **46 per cent actively distrust** the accuracy of AI tools against **33 per cent** who trust it. Three per cent report highly trusting it.

What it means for us. Documents are the top use, not a side use. And "almost right, but not quite" is the exact condition the change queue exists for.

§1 4 The instruction file is now a mass-market artefact

[0] Measured today against GitHub's code search. These are GitHub's own approximate counts, at repository root, public repositories only.

FILE	COUNT
AGENTS.md	about 547,840
CLAUDE.md	about 527,360
.kiro/requirements.md	about 33,216
GEMINI.md	about 27,072
llms.txt	about 21,120
.cursorrules	about 20,096

A correction I owe, because I got this wrong first time. Our plan says AGENTS.md is used by "over 60,000 projects". I initially wrote that the plan was stale by nine times. It is not.

- [M] agents.md **still says** "used by over 60k open-source projects" today. The plan quotes its source correctly.
- The two numbers measure different things. Theirs is projects, and it is their own claim. Mine is files at repository root in the public half of GitHub, counted approximately, including forks.
- **What is fair to say:** the floor is 60,000 projects and GitHub's own index holds something like half a million files. The direction is one way, and the plan should carry both numbers with their methods rather than one number with neither.

[M] And the demand around it is the loudest single artefact in this whole round. GitHub issue anthropics/claude-code#6235, asking Claude Code to read AGENTS.md, carries **6,644 reactions and 396 comments**.

§1 5 Spec-driven development is enormous in attention and thin in practice

[0] Star counts read from the GitHub API today.

REPOSITORY	STARS
anthropics/skills	176,889
github/spec-kit	137,619
modelcontextprotocol/servers	90,423
Fission-AI/OpenSpec	68,919
bmada-code-org/BMADA-METHOD	53,157
kepano/obsidian-skills	48,515
openai/agents.md	24,432

Against that, public repositories actually carrying the Spec Kit file layout: about **4,736**.

Read those two together before betting on it. One of the most-starred repositories on GitHub, and almost nobody has the files on disk in public. Either people star it and never run it, or they run it privately, or the search count is unreliable. I cannot tell which from here.

[0] **My reading, and it is a reading, not a finding.** The idea of writing a spec for an agent is enormously popular and the practice is still thin. That is the best moment to sell a nicer way to do it, and the worst moment to bet the company on it.

§1 6 The markdown file in the repository is beating the purpose-built mechanism

[M] Vercel published an eval on 29 January 2026 under the title "AGENTS.md outperforms skills in our agent evals". It reached **524 points and 196 comments** on Hacker News, one of the highest-scoring agent-tooling posts in the whole sweep.

SETUP	PASS RATE
Baseline, no documents	53%
A skill, default behaviour	53%
A skill, with explicit instructions to use it	79%
An 8 KB documents index inside AGENTS.md	100%

Two more lines from the same page. "In 56% of eval cases, the skill was never invoked." And their own reaction: "This wasn't what we expected."

Handle with care. It is one vendor's eval, on one framework's documentation, and the page does not say how many cases sit behind each rate. Quote the direction, not the decimal.

What it means for us. The plain markdown file in the repository beat the purpose-built mechanism designed to replace it. We sell the editor for that file.

Four products landed on or beside our position between June and September 2026. Three are not a commercial threat. The fourth shipped two days ago and is funded.

PART TWO. SOMEBODY IS ALREADY STANDING ON OUR GROUND

§2 1 Ritemark took our sentence

[M] Home page headline, verbatim: **"Markdown editor with AI agents."** Subheadline: "Claude, Codex, and Gemini work alongside you in a visual editor. They read your files, edit them, create new ones. You stay in control."

- On files: "No upload, no indexing, no copy-paste. The agent works on the same Markdown files you do, right where they already live."
- Closest to our projection law: "Drafts in ~/docs/q2-strategy/ stay in ~/docs/q2-strategy/."
- On price: "Completely free to download and use. No subscription, no account, no hidden fees."
- macOS and Windows. Open source.

What they skipped: no MCP anywhere on their comparison page, no change queue, no accept or reject, no byte-range splice, no refusal, no web, no phone, no sharing, no collaboration. Their control story is that the terminal is visible, which is not review.

What it costs us. The headline is gone. "A markdown editor with AI agents" is now a commodity claim and we should stop reaching for it.

§2 2 OpenMarkdown built the agent bridge, and handed us the design

[M] Launched 18 July 2026, 190 upvotes on Product Hunt. 61 stars. Free. macOS and Linux get the agent bridge, Windows does not yet. Its own line on models: "No AI inside. That's the point. It runs on the agent you already have."

Its eight MCP tools, verbatim: `open_file`, `open_folder`, `reveal`, `get_context`, `execute_command`, `read_section`, `write_section`, `wait_for_change`.

How it stops the agent clobbering you, verbatim: writes are "section-scoped with optimistic concurrency", and "if you and the agent touch the same section at the same time, the agent's write comes back as a `CONFLICT` so your unsaved edit is never clobbered".

Three things to take from it.

- Our refusal law was arrived at independently by somebody else, and shipped. Good evidence the law is right. Also evidence it is not by itself a moat.
- `wait_for_change` is the one idea in the set we have not had. The agent parks and waits for the person to edit, instead of finishing and leaving. That turns the editor into the meeting point rather than the drop box.
- `reveal` scrolls the person's window to the heading the agent is talking about. Shared attention with no collaboration engine behind it.

§2 3 OpenKnowledge gave us our own marketing line, by accident

[M] Posted to Hacker News as a Show HN on 25 June 2026, **381 points and 173 comments**, the highest-scoring markdown launch of the year. Free, open source, from a Y Combinator company. It syncs with a conflict-free replicated data type, a CRDT, which is the approach we ruled out.

The top comment that matters, verbatim from a stranger who has never heard of us:

"Nice. the frontmatter question is the one i'd want answered before trusting it: when an agent edits a file does it round-trip YAML frontmatter and nested code fences cleanly, or does that stuff get mangled? every 'wysiwyg markdown' tool i've tried falls apart there."

The founder's answer, verbatim: "we do optimistic parsing of it, haven't seen any issues with it."

Read that again. The single highest-value question asked at the top of our category was answered with a hope. We have 8,513 byte-pinned files and a corpus gate that exits 1 on one changed byte. **We can answer it with a test.**

And the phrase is already our name.

§2 4 Zed Delta went to public beta two days ago, and it is the serious one

[M] Published 16 September 2026 by Nathan Sobo: "Today, we're launching the public beta of Delta, a multiplayer environment for coding with agents and reviewing what they build."

Their problem statement is ours with one word changed. "But with agents generating so much code, the diffs we're asking each other to review have mushroomed." And: "Why should your teammate's agent have to guess how you got there?"

- **They left git's review surface behind.** "Last week, we crossed a key milestone: we disabled pull requests on Delta's own repository."
- **They store the work between commits.** DeltaDB "extends Git's content-based versioning with incremental versions based on deltas. It records edits between commits alongside messages from humans and agents." The commit stays the checkpoint. The deltas are what happened in between.
- **A review carries the agent's context with it.** "A review guides you through your branch's changes with access to the original agent's context."

Why this matters more than the other three. Ritemark and OpenMarkdown are single-player desktop tools. Zed is a funded team, shipping the review-the-agent product, with a storage model built for it, and they are dogfooding it hard enough to switch off pull requests.

What it does not touch. It is for code, in a code editor, for developers. It is not a document tool, there is no Doc mode, no publishing, no phone, and no non-developer anywhere near it.

And note what their model says. Recording the work between commits, with the human and agent messages attached, is the same instinct as T11. They did it at the storage layer. We would do it at the document layer. [0] Our settled position against conflict-free replicated data types now has three shipped products on the other side of it, and the rebuttal the plan already owes for Zed is overdue.

§2 5 What none of them did

[M] The paid incumbents, read from their own pricing pages today: Confluence with Rovo credits, Dropbox Dash whose four tabs are Find, Answer, Organize and Share with no write tab, Slite at twenty dollars a seat, Notion where the agent starts at Business, Craft where AI is credits.

Every one of them shipped retrieval, chat and generation. Not one shipped an accept step on a machine-written change.

- The free tools took "agents can edit your markdown".
- The paid tools took "ask your documents a question".
- **Nobody took "approve what the machine wrote."**

[0] And note the second hole. Every one of them meters the AI. The customer is billed for the thing that generates work, and not for the thing that reviews it.

Each one gives the demand with its number, who ships it, the effort, and a verdict. Effort is small for days, medium for a week or two, large for a month or more.

Tier one. These change the plan

§13 T1. The MCP server moves out of Later, and it proposes rather than writes

The demand.

- [0] Obsidian's live plugin registry holds **7,706 plugins**. About **95** of them exist to join an agent to a vault. The largest, `realclaudian`, has **2,135,277 downloads**. Six of the top seventeen exist for one purpose, to expose a person's notes over MCP: `vault-as-mcp`, `mcp-tools`, `semantic-vault-mcp`, `cli-rest-mcp`, `mcp-rest`, and the REST API extension.
- [M] `Aider-AI/aider#3314`, titled simply "MCP SUPPORT", has **196 reactions**. A second asking the same thing has 137.
- [M] Both new rivals in part two built one. It is what got them written about.

Section 6 of our own plan sets the rule: a capability that is a top add-on in three or more ecosystems ships built in. Ninety-five people have shipped this add-on in the one ecosystem we copy our feature list from. It is in our last column.

It is technically open to us. [M] The MCP specification of 28 July 2026 defines a **Streamable HTTP** transport alongside stdio, and says "Protocol semantics are identical on every transport". Authorization over HTTP is OAuth 2.1, where "A protected MCP server acts as an OAuth 2.1 resource server". So a hosted editor can serve agents. A desktop editor can only serve the agent on that laptop.

The shape, and this is the whole point. [M] Every write-capable document server in the official registry is a `write_file` or an `edit_file`. None has a queue, a proposal or a refusal.

- Documents as **Resources** over `file://` URIs, with subscribe for live updates.
- **Three tools only.** One proposes a splice into the change queue and returns a proposal id. One reports the queue. One locates a byte range and says plainly when the range is ambiguous.
- **Scopes map onto our tiers.** `documents:read` costs nothing. `documents:propose` writes to the queue. `documents:write` is the one we may never grant.
- **And one more scope, because people are asking for it.** [M] Cursor's forum carries a request to "lock certain files from my codebase in Cursor AI, preventing them from being accidentally modified by the AI". Nothing in the field ships it: permission systems gate the **tool**, not the **file**. A per-document lock that says an agent may read this and may not write it is one flag in our data model and one check in the propose path.

Effort. Medium for read-only over Resources. Large for propose-and-queue, because the interesting part is the refusal contract, not the transport.

Verdict. Must-have, and it moves into the build. Do not ship another `edit_file`.

§14 T2. The instruction-file set becomes the headline of S11, not a tab in it

The demand.

- [M] 6,644 reactions on anthropics/claude-code#6235 . The loudest artefact in the round.
- [M] agentsmd/agents.md#91 on the duplication, 134 reactions, listing four different global paths for four tools. #9 on directory support, 109 reactions, listing seven separate per-tool rules folders. #11 on imports, 70 reactions, every one a thumbs up.
- [0] An ordinary team on Claude Code, Cursor and Copilot keeps four files for one set of rules. Add Windsurf and Zed and it is six or seven.

What is shipped today. A shell command. The agents.md site's own advice is a symlink. Claude's memory page offers `ln -s AGENTS.md CLAUDE.md` and notes it needs Administrator rights on Windows.

Nobody ships a view that shows the whole set at once and says when two copies have drifted apart.

What we build. List every instruction file in the repository. Say which tool reads each. Flag a file that is a copy rather than a link. Diff two copies that have drifted, which our byte-exact engine does already.

And there is already a specification for what to check. [M] A peer-reviewed catalogue, "Configuration Smells in AGENTS.md Files", measured 100 popular repositories carrying one of these files. Its own summary box: "We detected at least one smell in 91 agent configuration files. Thus, only nine files were found to be smell-free."

Its six named defects are a ready-made problems panel: context bloat, skill leakage, lint leakage (rules a linter already enforces), blind references to files that are not there, and two more.

The honest caveat, and it changes what the screen may claim. [M] The literature is split on whether these files help at all.

- **Against.** arXiv:2602.11988, revised 18 September 2026: "we find that providing context files does not generally improve task success rates, while increasing inference cost by over 20% on average."
- **Against, independently.** arXiv:2607.27250: "Context strategy does not measurably move correctness on either agent", across 288 evaluated runs.
- **For, on a different axis.** arXiv:2601.20404 measured "a lower median runtime (28.64%) and reduced output token consumption (16.58%)".
- **And Vercel's eval from 1.6** took a pass rate from 53 to 100 per cent.

[0] **Where that resolves.** The first paper says it plainly: context files "are useful for specifying non-standard coding practices", and the repository overview is the part that does not pay. Vercel's win came from documenting an interface published after the model was trained.

So the file earns its tokens when it carries something unguessable, and wastes them when it restates the repository. We may say the screen helps you write a correct file. **We may not say it makes your agent smarter.** Two independent studies say it does not.

Effort. Medium. Detection and listing is days. The drift diff is a few days more. The six defect checks are mostly a line count and a git commit count.

Verdict. Must-have. Our S11 is a health panel today. This is the screen. And the empty surface is real: [M] twelve linters exist for these files, all command line, and the one editor extension has 239 installs after seven months.

§15 T3. Open a spec folder as one document

The demand, and this is the cleanest hole in the whole round.

[M] As of 18 September 2026 there is no product that gives a spec-driven-development specification a good editing surface outside a code editor. The three shapes that exist:

- A sidebar in a code editor. The best of them, specs.md, says in its own documentation: "When you click a document, story, standard, or artifact, specsmd asks your operating system to open that local file." That is the ceiling. It cannot show you the document.
- A terminal. Kiro's review pane, OpenSpec's `openspec view`, specs.md's own terminal dashboard.
- A proprietary desktop app, Kiro's, at twenty to two hundred dollars a person a month.

And the strongest single signal in that lens. [M] Amazon Web Services decided that reviewing a spec needs line-anchored comments badly enough to build a full document reader with vim keys **inside a command line**. From Kiro's own docs: "Press `Ctrl+X` to read the document in place and stage comments against the lines you want changed. Returning to the checkpoint keeps your comments staged, and answering the checkpoint question sends them all to the agent as one revision request."

A commercial team shipped a worse version of Google Docs comments in a terminal, because the spec is a markdown file in a repository and only code tools can reach it.

And the users of the biggest spec tool have asked for it, in the issue title. [M] `github/spec-kit#1191`, titled "Spec-Driven Editing Flow: Can't Easily Update or Refine Existing Specs", carries **115 reactions and 8 comments**. From the body: "Multiple users find it difficult to update, refine, or iterate on existing specs in Spec Kit without creating new branches and redundant specification artifacts", and "Workarounds (manual file edit, running `/speckit.clarify`, copying files) are unclear, error-prone, and not universally documented."

It is a hub issue. It links ten more, opened independently, including "Right way to change Spec after `/implement`" and "How to keep specs consistent and up-to-date with spec-kit?".

The file layouts are settled and public. `specs/<###-feature>/spec.md` and `plan.md` and `tasks.md` for Spec Kit. `.kiro/specs/<feature>/requirements.md`, `design.md`, `tasks.md`. `openspec/changes/<id>/proposal.md`. None of them invented a container format. They are all folders of loose markdown.

What we build. Recognise the folder by its path. Stitch the files of one feature into one reading surface. Doc mode over the spec. Tasks view over `tasks.md`. Mermaid over `design.md`. Share by link so a product person can read the requirements without a checkout.

Effort. Medium, and most of the parts are already in the plan. The new work is recognising the folder shape and stitching several files into one surface.

Verdict. Must-have, and it is the cheapest big win in the round. We do not own the format, we do not change a byte, and the files are already on tens of thousands of machines.

§16 T4. The published page serves a markdown twin and an `llms.txt`

The demand. Part 1.1. Two to one, and 83 per cent of it arriving by markdown route.

Why it is nearly free for us and awkward for everyone else. Our source of truth is already the markdown file. Serving the raw bytes at `page.md` is a route, not a converter. Every rival has to generate markdown out of blocks. We would be serving the original.

The caveat, and it is a real one. [M] Ahrefs analysed 137,000 domains and reported that **28 per cent publish an llms.txt and 97 per cent of those files got zero traffic** in May 2026.

Both numbers are true, and they are about different populations. Ahrefs measured the general web, where nobody is looking for a documents file. Mintlify measured documentation sites, where agents actually go, and there the markdown route carries 83 per cent of agent traffic. Do not promise a marketing lift. Promise that a page you publish is readable by the thing reading it.

Effort. Small. A content negotiation header, a `.md` route, a generated `llms.txt`, and a link to it from the markdown output. Days.

Verdict. Must-have for the published page, on the readability argument rather than the traffic one. It is the one place where being file-first is a mechanical advantage rather than a philosophical one.

§17 T5. Stamp the accept into the front matter, using Google's field names

The demand. [M] Google Cloud published the **Open Knowledge Format** on 13 June 2026: "a vendor-neutral, agent- and human-friendly standard", which "represents knowledge as a directory of markdown files with YAML frontmatter".

Then v0.2, on 25 July 2026, added exactly what we cut. Its five questions, verbatim: "What was this created from? (provenance)", "How much should I trust it? (trust)", "Is it still true? (freshness)", "Is it the current version? (lifecycle)", "Was this number produced the way we said it must be? (attestation)".

The new front matter keys:

KEY	WHAT IT HOLDS
<code>generated: { by, at }</code>	how the content was produced, and when it last meaningfully changed
<code>verified: [{ by, at }]</code>	a list of independent confirmations
<code>sources</code>	what it came from
<code>stale_after</code>	when to stop trusting it
<code>status</code>	moves through draft, stable, deprecated

Why this matters more than it looks. We dropped authorship marking on 17 September because Almanac shipped read receipts and died. That reasoning was about a **product feature**. This is a **file field**. It survives in the file, it is portable, we do not have to invent it, and we do not have to defend it.

What we build. When a person accepts a change in the queue, write `verified: [{by, at}]` into the YAML block. That is a two-line splice into the part of a markdown file we already promise not to mangle. **The change queue becomes the thing that produces the trust signal.**

Effort. Small for the stamping. Medium for a real validator wired to the problems panel.

Verdict. Must-have for the stamping, good-to-have for the validator.

§18 T6. The problems panel becomes the bundle, and the bundle is the pitch

The demand, as a count rather than an argument. [0] To get every check in this research today, a person installs eleven things, through four package managers, with nine configuration files.

#	INSTALL	CONFIGURATION IT NEEDS
1	<code>markdownlint-cli2</code>	<code>.markdownlint-cli2.jsonc</code>
2	the Vale binary	<code>.vale.ini</code>
3	Vale style packages	a <code>styles/</code> directory
4	<code>cspell</code>	<code>cspell.json</code> and a project dictionary
5	<code>lychee</code>	<code>lychee.toml</code> and a cache file
6	<code>remark-lint-frontmatter-schema</code>	<code>.remarkrc</code> and a JSON Schema
7	Harper	none, which is why it is the pleasant one
8	<code>drift</code>	an anchor in every document's front matter
9	<code>doc-freshness-checker</code>	<code>.doc-freshness.config.js</code> , and Node 24
10	<code>textlint</code> and one install per rule	<code>.textlintrc</code>
11	an editor extension for each one you want to see while typing	per-extension settings

Who ships the bundle. Nobody. Mintlify ships two of the eleven at \$450 a month.

The decisive advantage is not that we can run these checks. It is that **the file is already open**. Every one of these tools makes the writer wait for a build, then reads them a line number for a file they have closed.

Effort. Small each for the lint-equivalent, spelling, links, front matter schema and readability. Medium for style rules and for staleness, which is T10.

Verdict. Must-have. The pitch is one sentence. *Eleven tools, four package managers, nine configuration files, or open the file.*

§19 T7. Front matter is the most-wanted thing on the largest markdown forum, and it is our name

The demand. [M] The top-liked requests of all time in Obsidian's feature requests category.

REQUEST	OPENED	LIKES ON THE TOPIC	VIEWS
Use the H1 or the YAML <code>title</code> as the display name	2020-05-28	712	54,840
Support multi-level, nested YAML	2023-07-27	790	30,559
Let the user set how dates and times display		197 on the opening post	
Properties Wrangler		210	

Four of the top fourteen requests of all time are about front matter. The nested-YAML one has **182 posts over three years** and was still active on 24 August 2026.

Why this is ours. A properties editor that cannot represent nested YAML has to do something when it meets a file that contains it. That something is a round-trip defect, and refusing to mangle a file is the one thing we promise. Our product is called frontmatter.

Effort. Medium. A properties editor that round-trips nested YAML is real work, and it is work our engine's premise already commits us to.

Verdict. Must-have. The largest audience in markdown has been asking for this for six years.

Tier two. Good to have, and the evidence is strong

§20 T8. Suggesting mode, over the queue we already have

The demand is a build count, not a vote count. [0] Five people independently built a CriticMarkup suggesting mode for Obsidian between **17 April and 7 July 2026**, none of them apparently aware of the others, three naming AI in the description and one naming Claude Code.

PLUGIN	DOWNLOADS	WHAT IT SAYS IT DOES
<code>track-changes</code>	1,768	"Review CriticMarkup suggestions in a side panel. Accept, reject, or reply."
<code>redline</code>	834	comments anchored to paragraphs, in a sibling <code>.review.md</code>
<code>review-comments</code>	763	"Notion-style review comments stored as CriticMarkup."
<code>review-critic</code>	735	review and CriticMarkup comments
<code>live-coedit</code>	101	"it proposes edits, you review them as track changes"
<code>review-comments-ai</code>	97	threads live in the note so Claude Code can reply in place

The serious prior attempt tells us why nobody has won. [M] Commentator, by one person, aiming to "recreate Word's/Google Docs' suggestion and comment modi within Obsidian". Its announcement thread has **14,535 views and 75 likes**. The repository was created in February 2023 and is **still a beta plugin**, not in the store, three and a half years on.

The lesson. The hard part is not the idea and not the syntax. It is being a product rather than a plugin. A plugin cannot own the edit path, so it fights the host for every keystroke. **We own the edit path.** This is the classic case where the thing that is impossible as an extension is natural as the application.

One hard line, and it is measured. [0] CriticMarkup's substitution mark collides with GitHub strikethrough. `{~quick~>slow~}` renders in both marked and markdown-it as struck-through text reading `quick~>slow` wrapped in stray braces. The reader does not see a proposed edit. They see corruption. Tested against three CommonMark parsers in this session.

So: speak CriticMarkup, do not store in it. Hold the proposal outside the file, splice on accept, offer export for anyone handing the file to those six plugins or to pandiff.

Effort. Medium for the core. The queue, the splice engine and the diff exist. The new parts are the suggestion overlay and the annotation panel.

Verdict. Must-have. It is the one place where we can be the first shipped answer to a question five people asked this year.

§21 T9. Comments anchored to a byte range, batched into one revision request

The demand. T3's Kiro finding. A person needs to say "this line is wrong" on a specific line of a generated document, and have the agent revise from those comments as one batch.

Why it is close to native for us. A comment anchored to a byte range plus a quoted excerpt is the same addressing our splice engine already does, under the same refuse-when-ambiguous rule.

Effort. Medium. The anchoring surviving edits is the real work, and our engine already does it.

Verdict. Good-to-have, high conviction.

§22 T10. Staleness: anchor a paragraph to code, and say when the code moved

The demand. [M] Fiberplane published "We built a linter for documentation rot" on **25 March 2026**, opening with: "we've been writing some specs and we noticed they go stale quickly". On the market gap: "Existing approaches all focus on keeping code examples in sync, not the prose that describes them." On why not to use a model: "burning tokens every commit to answer 'did this doc go stale?' is expensive for what should be a trivial check."

How it works, because the mechanism is the finding. An anchor is a path, an optional symbol and a git sha, living in the front matter or inline. `drift check` resolves the baseline commit, parses both versions and compares a normalised syntax-tree fingerprint, so reformatting does not raise a false alarm.

And somebody is charging for it. [M] Promptless prices documentation freshness at **\$500 a month** for up to 200 pages, rising to \$2,000 to \$4,000 for 2,000 to 5,000 pages. It carries a **second product line for agent instruction files**, priced by instruction count.

Nobody puts it in an editor. Drift is a command line tool and a CI gate. A person editing `docs/auth.md` gets no signal at all that the code under the anchor moved. They find out hours later when somebody else's build fails.

Why this is the one that keeps people. The most valuable thing a problems panel can say about an agent-written document is **"the code this paragraph describes changed on Tuesday and nobody came back."** Everything else in that panel is cosmetics next to it.

Effort. Medium. Comparing file bytes at the baseline commit catches most of it in days. Symbol-level anchors need a grammar per language.

Verdict. Must-have. It needs no model, no network and no server, and there is a company charging five hundred dollars a month for the same answer.

§23 T11. Rewind the document and the agent's context together

The demand. [0] Measured today across the agent tools' issue trackers, thumbs-up counts:

REACTIONS	ISSUE	TITLE
452	<code>openai/codex#9203</code>	Please make "/undo" back
233	<code>openai/codex#2998</code>	IDE-integrated diff / approval
221	<code>openai/codex#11626</code>	CLI: Add /rewind checkpoint restore that reverts both chat context and Codex-applied code edits
41	<code>Aider-AI/aider#649</code>	Add option to force the AI to ask the user to confirm each change before doing it

947 reactions across four issues in two repositories, all saying one thing. Let me see what the agent did, approve it, and put it back if I do not like it.

Note what #11626 asks for that we do not do. Rewinding **the conversation and the file together**. Our history restores the document. It does not restore the state of the thing that wrote it.

Effort. Medium, and only if we hold the conversation, which today we do not.

Verdict. Good-to-have. The strongest external evidence the change queue has ever had, from people who have never heard of us. Worth one screen, not a phase.

§24 T12. Shared attention: reveal, and wait for the person

The demand. OpenMarkdown built both and got written about for it. Neither needs a collaboration engine.

- **Reveal.** The agent scrolls your window to the heading it is working on. You read the rendered page instead of a terminal diff.
- **Wait for change.** The agent parks and waits for you to edit, then resumes.

Effort. Small each, once T1 exists. They are two more tools on the same server.

Verdict. Good-to-have. Cheap, and it is the difference between a drop box and a meeting point.

§25 T13. Review from the phone

The demand. [P] Follows from 1.2 and T11. The agent finishes at two in the morning. Somebody has to approve it.

- We already have a phone layout for every screen.
- [0] Nobody has built mobile review of an agent's **document** work. The mobile approval pattern exists for code.

Effort. Small, given the phone layouts are already costed.

Verdict. Good-to-have, and it is a demo. It is the single most showable thing in this list.

§26 T14. A live size readout on an instruction file, and nothing more

The demand, honestly reported, including the part that cuts against it. [M] The measured complaint exists: a fresh Claude Code session was measured at 40,613 to 69,534 baseline tokens across ten sessions. But that issue has **one thumbs up**, against 6,644 on the AGENTS.md request. And the person who measured it tried trimming the instruction file and reported it "none moved this baseline more than a few percent".

So build the honest version. Show bytes, lines and an estimated token count against the published limit for that file type. Do not promise it fixes their session.

Effort. Small.

Verdict. Good-to-have, downgraded on the evidence. Ship the readout because it is cheap and true. Do not build a marketing claim on it.

§27 T15. Draft, then promote, as a visible state on the file

The demand. [M] Three strangers in one Hacker News thread, on a Show HN for an agent-maintained markdown wiki, independently asked for the same thing: a document an agent wrote should sit in a draft state until a person promotes it.

And Google shipped the field for it. T5's `status` key moves a document through draft, stable and deprecated.

Nobody ships it as a state in an editor. Everyone has drafts. Everyone has published. Nobody has *an agent wrote this, it is a draft until a person promotes it, and the file says so*.

Effort. Small, once T5 exists. It is the same front matter key and one filter on the tree.

Verdict. Good-to-have, high conviction. It is the change queue expressed as a property of the file rather than as a screen, which means it survives outside our app.

§28 T16. A multi-file change that arrives as a reviewable set

The demand. [M] Global search and replace across a whole workspace is on Obsidian's all-time request list and has never shipped. [R] Our own section 2 already lists it at 650 hearts as one of "the three that matter most". People also ask for an atomic changeset across several documents rather than a version per document.

Nobody ships a multi-file edit that arrives as a **reviewable set** you approve hunk by hunk before anything is written.

Why it is ours and nobody else's. Everyone else's replace writes and then offers undo. We have a queue and a splice engine that refuses when a range is ambiguous. A workspace-wide rename is the same queue with many more hunks in it.

Effort. Medium. The engine and the queue exist. The work is the match preview.

Verdict. Good-to-have, and strategically the most interesting item in tier two. It turns our refusal law from a safety feature into a capability nobody else can copy cheaply.

§29 T17. A command line, because in 2026 the thing typing the command is an agent

The demand. [M] Typora's advanced command line interface request is a long-running summary issue. [M] And Obsidian's chief executive publishes the skills that teach an agent to drive Obsidian's own command line, which 48,515 people have starred.

Nobody ships a markdown editor you can invoke from a script and wait on.

Effort. Small for open-and-wait. Medium if it grows export and window control.

Verdict. Good-to-have. It is the cheap half of T1, and it is the half that works before the server exists.

Tier three. Already in our own research, never in a screen

[R] These come from our own corpus, not from the outside world. Each cites where we already wrote it down.

GAP	WHERE WE SAID IT	COST
Global search and replace across the workspace	our own section 2 lists it in "The three that matter most", 650 hearts, and Obsidian still has not shipped it.	small
Rename a tag, heading or key across the vault, every hunk reviewable	86 likes on the linked request, and it was a costed row.	small
A refusal has a screen	two of our own decision cards ask for it, neither answered, although refusing is the whole differentiation.	small
Search has a screen	T11 of our own pain taxonomy. Retrieval at scale has no feature at all.	medium
Import from Evernote, OneNote and Apple Notes	T5 of the taxonomy, the refugee funnel.	medium
Offline says which documents you will still have, not just that you are offline	unmet need 7 in the taxonomy.	small
Database views over front matter	the 9 September briefing files this under "What is now table stakes".	medium
Right to left, system text scaling, high contrast	candidate decision 35. In no screen and no phase.	medium
An always-visible sync status	the gap map names it twice as the most concrete trust ask.	small
Whether a link recipient needs an account	never stated, and our two answers conflict.	none, decide it
A free public tool as a funnel	both our funnels sit behind sign-in.	medium

And four contradictions inside our own documents, which need deciding rather than building.

- The plan says it is written to K1 recommendation b, and the sentence it carries is option a.
- Free is capped at 50 documents, and the pilot recruits vaults of at least 200 files.
- The front door forbids anonymous editing, and the permission matrix grants it.
- Voice typing is a shipped Doc mode row and the research's clearest negative.

This is the uncomfortable half, and it is the more useful half.

PART FOUR. WHAT WE MUST STOP SAYING

§4 1 The change queue is not novel, and saying it is invites an easy rebuttal

[M] Google Docs already ships per-change accept or reject for AI edits. From Google's own help page: "To apply changes individually, click Accept suggestion", "To apply all changes, click Accept all", "To reject all changes, click Reject all."

The defensible claims are narrower, and all three are about durability and place.

- Ours applies to a change an **external agent** made to the file **while the editor was closed**, not only to what the built-in assistant proposes in the session.
- The decision is recorded in a version record that **outlives the accept**. [M] Once a Gemini suggestion is accepted, the text is ordinary text and the history credits the human.
- It works over a **markdown file that other tools own**.

[M] And note their own bypass. Google's side panel has an "Insert" action that writes generated text into the document with no review step at all. Two doors, one queue.

§4 2 We are not first to review markdown prose

[M] CodeRabbit has reviewed markdown prose since 2024. Do not claim the category.

§4 3 The headline is taken

Part 2.1. Ritemark is free and says it better than we would have. Our line has to be about the accept, not about the agent.

§4 4 Do not clone iA Writer's annotations

[M] iA Writer already built a byte-range-plus-hash authorship model, aimed at AI text, and published it. Their own refusal rule is ours: when the hash is invalid, "tools should warn users that annotations may be misplaced".

Their licence note, recorded verbatim because it is load-bearing: "While the format is open, avoid cloning our work. Draw inspiration from what we made. Change it. Improve it. Design it yourself."

Read and write their format for interoperability. Do not make it our storage. Also note their finding, which saves us the experiment: "The markup always got in the way when editing because every other word can have a different author."

§4 5 We retired an idea on a reason its own evidence does not carry

[M] Almanac did shut down. Their own farewell page says so: "We've made the decision to shut down Almanac on January 31, 2025."

But not for the reason we recorded. The page gives the reason in order, verbatim. A second product, Blaze, "has since taken off, growing to tens of thousands of users in a matter of months, and it's consumed all of our small team's time and energy". Then: "we've decided we don't want to offer a product in Almanac whose quality we no longer have the capacity to maintain or improve."

- **That is an attention story, not a feature verdict.** The same page says Almanac "has been adopted by many of the world's best remote companies".
- Our CLAUDE.md records that per-span read state was dropped on 17 September partly because Almanac shipped read receipts and shut down. **The shutdown is real. The causation is not supported.**
- Put alongside T5, where `verified: [{by, at}]` is now a field in a Google specification, the idea deserves one more look rather than a closed file.

One more thing to record. [M] There is a new and unrelated Almanac. `usealmanac.com` is a Y Combinator S26 company whose home page reads "Meet Almanac, The agent with a second brain", with a section headed "The wiki that self-updates". Anyone checking our citation will now land on it.

§4 6 The thing we should say instead

"The frontmatter question." A stranger named it at the top of our category, unprompted, and the category leader could not answer it. We have 8,513 byte-pinned files and a gate that fails on one changed byte.

Publish the round-trip result as a public claim and invite comparison. Effort: days. It is the cheapest marketing asset available to us and it is already built.

Nothing here changes what frontmatter is. It changes what ships first.

PART FIVE. WHAT THIS DOES TO THE PLAN

§5 1 The seven moves

1. **Move the MCP server out of Later.** It is the most-demanded add-on in the ecosystem we copy from, all three new rivals lead with it, and it is the only thing that makes a hosted editor better than a desktop one rather than merely different.
2. **Make S11 the instruction-file screen** rather than a health panel on one file. 6,644 reactions.
3. **Add spec-folder recognition to phase C.** It is mostly parts we are already building, and it is the one clean hole in the market.
4. **Add the markdown twin and lms.txt to phase D.** Days of work, measured twentyfold effect.
5. **Stamp the accept into the front matter,** using Google's field names. A two-line splice that turns the change queue into a portable trust signal.
6. **Widen the problems panel into the bundle.** Eleven tools, four package managers, nine configuration files, or open the file.
7. **Round-trip nested YAML properly,** because four of the top fourteen requests on the largest markdown forum are about front matter, and front matter is our name.

§5 2 What it costs

[0] My estimate, and it is an estimate: about **five to seven weeks** on top of the 27-week appetite, if T1 ships read-only first and the propose tools follow. T4 and T5 are days. T6 is mostly small pieces. T2, T3 and T7 are the real cost.

Where to take it from. The Later column is the release valve the plan already names. Kanban, table-to-chart and the community can wait behind this.

§5 3 The one question this raises for the founders

The plan sells an editor. These findings describe **the surface between a person and their agents**, which is a slightly different product with a slightly different buyer.

That is a K1 question, and it is yours. I have not moved anything in the plan. This document is the proposal.

- Nine research branches, each given the same evidence rules: open every page, never put a paraphrase inside quotation marks, record the URL and the date, mark inference and mark anything unverified.
- 141 findings, in `docs/research/2026-09-18/raw/`.
- Five direct measurements in this session: GitHub code search counts, GitHub issue reaction counts, the live Obsidian plugin registry, repository star counts, and a CriticMarkup collision test against three installed CommonMark parsers.

What this round did not do. No customer was interviewed. No prototype was tested. No price was tested. Reaction counts measure the people who file issues, not the market, and they are cumulative since the issue opened, so an old issue has had longer to gather them. I did not normalise for age.

Two numbers to handle with care.

- GitHub's code search returns an **approximate** total. Every figure came back suspiciously round. Quote them as orders of magnitude.
 - The Stack Overflow figures are **2025**. There is no 2026 AI page; the URL returns 404. Do not date them 2026.
-

Every page below was opened on 18 September 2026. The full findings, with the quoted strings, are in `docs/research/2026-09-18/raw/`.

The market numbers

- [Mintlify, State of Knowledge 2026](#)
- [Stack Overflow developer survey, AI section, 2025](#)
- [Ahrefs on llms.txt across 137,000 domains](#)
- [Vercel, AGENTS.md against skills](#)

The standards

- [Model Context Protocol, 2026-07-28](#)
- [Open Knowledge Format v0.1](#)
- [Open Knowledge Format v0.2, the trust fields](#)
- [AGENTS.md](#)
- [llms.txt](#)
- [CriticMarkup toolkit](#)
- [iA Writer, Markdown Annotations](#)

The products on our ground

- [Ritemark](#)
- [OpenMarkdown](#) and its [FAQ](#)
- [OpenKnowledge on Hacker News](#)

Spec-driven development

- [GitHub Spec Kit](#)
- [Kiro, the terminal spec review](#)
- [OpenSpec](#)

Checking a document

- [Fiberplane, a linter for documentation rot](#)
- [Promptless pricing](#)
- [Vale](#)

The papers

- [Configuration smells in AGENTS.md files](#), arXiv:2606.15828
- [Evaluating AGENTS.md](#), arXiv:2602.11988
- [Do context files help coding agents](#), arXiv:2607.27250
- [On the impact of AGENTS.md files on efficiency](#), arXiv:2601.20404

The demand counts

- [Support AGENTS.md](#), 6,644 reactions
- [A global user-level AGENTS.md](#)
- [Please make /undo back](#)
- [Rewind checkpoint restore](#)
- [MCP support in aider](#)
- [Commentator, the CriticMarkup attempt](#)
- [Markdown annotations a la iA Writer 7](#)

The corrections

- [Almanac's own farewell page](#)
- [Google Docs, write and edit with Gemini](#)